SQL Server CI/CD Deployment

Complete Step-by-Step Technical SOP

Document Version 1.0

Table of Contents

1. Prerequisites and Requirements

Overview

This document provides comprehensive step-by-step instructions for setting up an automated SQL Server CI/CD pipeline using GitHub Actions and self-hosted Windows runners.

Software Requirements

- Windows Server 2019 or later, or Windows 10 Pro (or higher)
- SQL Server 2019 or later installed
- PowerShell 5.1 or higher
- Git for Windows installed
- sqlcmd command-line utility (installed with SQL Server)

Network Requirements

- GitHub.com account
- Network connectivity from Windows machine to SQL Server
- Network connectivity from Windows machine to GitHub

Repository Requirements

- GitHub repository created and initialized
- Repository must be public or private (you have access)
- Git installed and configured locally

Verify Prerequisites

Check PowerShell version:

```
$PSVersionTable.PSVersion
```

Should show version 5.1 or higher.

Check Git installation:

```
git --version
```

Should show installed version.

Check sqlcmd availability:

```
sqlcmd -?
```

Should display sqlcmd help documentation.

2. Initial Setup and Configuration

Step 2.1: Create Local Working Directory

Create a local directory for your repository:

```
cd F:
mkdir GITHUB
cd GITHUB
```

Step 2.2: Clone Repository

Clone your GitHub repository:

```
git clone https://github.com/YOUR_USERNAME/YOUR_REPO.git
cd YOUR_REPO
```

Replace YOUR_USERNAME and YOUR_REPO with your actual GitHub username and repository name.

Example:

```
git clone https://github.com/PMSQLDBA/SQL-CICD-DEMO.git
cd SQL-CICD-DEMO
```

Step 2.3: Create Scripts Folder

Create the Scripts folder where SQL files will be stored:

```
mkdir Scripts
```

Step 2.4: Verify Directory Structure

Check that your directory looks like this:

```
dir
```

Expected output:

```
Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----               [date/time]                  .git
d-----               [date/time]                  Scripts
```

3. GitHub Repository Setup

Step 3.1: Create .gitignore File

Create a .gitignore file to exclude unwanted files:

```
notepad .gitignore
```

Add these lines:

```
runners/
.env
.credentials
*.swp
*.swo
*~
```

Save and close the file.

Step 3.2: Commit .gitignore

```
git add .gitignore
git commit -m "Add gitignore"
git push origin main
```

Step 3.3: Create Initial SQL Script

Create your first SQL script in the Scripts folder:

```
notepad Scripts\Users.sql
```

Add SQL code:

```
IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'Users')
CREATE TABLE Users (
    UserID INT PRIMARY KEY IDENTITY(1,1),
    UserName NVARCHAR(100) NOT NULL,
    Email NVARCHAR(100) NOT NULL,
    CreatedDate DATETIME DEFAULT GETDATE()
)
```

Save the file.

Step 3.4: Commit SQL Script

```
git add Scripts\Users.sql
git commit -m "Add Users table script"
git push origin main
```

4. SQL Server User Creation

Step 4.1: Open SQL Server Management Studio

Launch SQL Server Management Studio and connect to your SQL Server instance.

Step 4.2: Create Login

In the query window, execute:

```
CREATE LOGIN Github_CICD_User WITH PASSWORD =
'Your_Secure_Password_Here'
```

Replace 'Your_Secure_Password_Here' with a strong password. Use uppercase, lowercase, numbers, and special characters.

Execute the query (Ctrl+E or click Execute button).

Step 4.3: Create User

Execute:

```
CREATE USER Github_CICD_User FOR LOGIN Github_CICD_User
```

Step 4.4: Grant Permissions

Execute:

```
ALTER ROLE db_owner ADD MEMBER Github_CICD_User
```

This grants the user full database permissions. For production, consider more restrictive permissions.

Step 4.5: Verify User Creation

Execute:

```
SELECT name FROM sys.server_principals WHERE type = 'S' AND name
LIKE '%CICD%'
```

You should see Github_CICD_User in the results.

Step 4.6: Record Credentials

Write down the following information for later use:

- Login: Github_CICD_User
- Password: Your_Secure_Password_Here
- Server: pmsqldba (or your server hostname)
- Database: DBAScripts (or your database name)

5. GitHub Secrets Configuration

Secrets store sensitive information encrypted in GitHub. They are only accessible within workflows.

Step 5.1: Go to Repository Settings

Open your browser and go to:

```
https://github.com/YOUR_USERNAME/YOUR_REPO/settings/secrets/actions
```

Replace YOUR_USERNAME and YOUR_REPO with your actual values.

Example:

```
https://github.com/PMSQLDBA/SQL-CICD-DEMO/settings/secrets/actions
```

Step 5.2: Create SQL_SERVER Secret

Click "New repository secret".

- Name: SQL_SERVER
- Value: pmsqldba (your SQL Server hostname)

Click "Add secret".

Step 5.3: Create SQL_DATABASE Secret

Click "New repository secret" again.

- Name: SQL_DATABASE
- Value: DBAScripts (your database name)

Click "Add secret".

Step 5.4: Create SQL_USERNAME Secret

Click "New repository secret" again.

- Name: SQL_USERNAME
- Value: Github_CICD_User

Click "Add secret".

Step 5.5: Create SQL_PASSWORD Secret

Click "New repository secret" again.

- Name: SQL_PASSWORD
- Value: Your_Secure_Password_Here (the password you created earlier)

Click "Add secret".

Step 5.6: Verify Secrets

After creating all four secrets, you should see all of them listed.

All secrets are now stored securely. They appear masked in workflow logs.

6. Workflow File Creation

Step 6.1: Create Workflow Directory

In PowerShell, create the workflow directory:

```
cd F:\GITHUB\SQL-CICD-DEMO
mkdir -p .github\workflows
```

Step 6.2: Create Workflow File

Create the deployment workflow:

```
notepad .github\workflows\deploy-sql.yml
```

Step 6.3: Add Workflow Content

Paste this complete workflow:

```
name: SQL Server Deployment

on:
  push:
    branches:
      - main
    paths:
      - 'Scripts/**'
  workflow_dispatch:

jobs:
  deploy:
    runs-on: self-hosted
    steps:
      - uses: actions/checkout@v4
      - name: Test Connection
        shell: powershell
        run: sqlcmd -S "${{ secrets.SQL_SERVER }}" -d "${{
secrets.SQL_DATABASE }}" -U "${{ secrets.SQL_USERNAME }}" -P "${{
secrets.SQL_PASSWORD }}" -C -Q "SELECT @@VERSION;"
      - name: Deploy Scripts
        shell: powershell
        run: |
          Get-ChildItem '.\Scripts' -Filter '*.sql' | ForEach-
Object {
            Write-Host "Executing: $($_.Name)"
            sqlcmd -S "${{ secrets.SQL_SERVER }}" -d "${{
secrets.SQL_DATABASE }}" -U "${{ secrets.SQL_USERNAME }}" -P "${{
secrets.SQL_PASSWORD }}" -C -i $_.FullName
```

```
            if ($LASTEXITCODE -ne 0) { exit 1 }
        }
```

Save the file (Ctrl+S, then close).

Step 6.4: Understand the Workflow

The workflow contains:

- name: Display name of the workflow
- on: Trigger conditions (when workflow runs)
- jobs: Workflow tasks
- deploy: Job name
- runs-on: self-hosted: Runs on your Windows machine
- steps: Individual actions

Step 6.5: Commit Workflow File

```
git add .github\workflows\deploy-sql.yml
git commit -m "Add SQL deployment workflow"
git push origin main
```

7. Self-Hosted Runner Installation

The self-hosted runner is a Windows application that listens for jobs from GitHub and executes them.

Step 7.1: Create Runner Directory

Create a separate directory outside your repository for the runner:

```
cd F:\
mkdir runners
cd runners
```

The runner must be in a separate directory from your repository.

Step 7.2: Download Runner

Download the GitHub Actions runner for Windows x64:

```
Invoke-WebRequest -Uri
https://github.com/actions/runner/releases/download/v2.336.0/acti
ons-runner-win-x64-2.336.0.zip -OutFile runner.zip
```

This downloads a 103 MB file. Wait for completion.

Step 7.3: Extract Runner

Extract the downloaded zip file:

```
Expand-Archive runner.zip
```

Step 7.4: Verify Extraction

Check that files were extracted:

```
dir
```

You should see:

```
config.cmd
run.cmd
bin/
external/
_diag/
runner.zip
```

8. Runner Registration and Configuration

Step 8.1: Generate Registration Token

Go to your repository runner settings:

```
https://github.com/YOUR_USERNAME/YOUR_REPO/settings/actions/runners
```

Click "New self-hosted runner".

Select: Windows and x64

GitHub displays setup instructions with a registration token. The token is valid for 1 hour only.

Copy the token (looks like: BQQJKA2VAXUFGTIRG5UN7GTKOA3O2).

Step 8.2: Register the Runner

In PowerShell, run the configuration command:

```
cd F:\runners
cmd /c config.cmd --url
https://github.com/YOUR_USERNAME/YOUR_REPO --token
YOUR_TOKEN_HERE
```

Replace YOUR_USERNAME, YOUR_REPO, and YOUR_TOKEN_HERE with your actual values.

Step 8.3: Follow Configuration Prompts

The script asks several questions:

Runner group: Press Enter for "Default"

Runner name: Press Enter for auto-generated name or type custom name

Additional labels: Press Enter to skip

Work folder: Press Enter for "_work"

Run as service: Press Enter for "No" (runs interactively)

Step 8.4: Verify Configuration

After configuration completes, verify files were created:

```
dir
```

You should see new files like .runner, .credentials, and .credentials_rsaparams.

9. Testing the Pipeline

Step 9.1: Start the Runner

Launch the runner:

```
cd F:\runners
.\run.cmd
```

Expected output:

```
√ Connected to GitHub
Current runner version: '2.336.0'
Listening for Jobs
```

The runner is now listening for workflow jobs. Keep this window open.

Step 9.2: Trigger a Deployment

In a new PowerShell window, make a change to trigger the workflow:

```
cd F:\GITHUB\SQL-CICD-DEMO
echo "-- Deployment test" >> Scripts\Users.sql
git add Scripts\Users.sql
git commit -m "Test SQL deployment"
git push origin main
```

Step 9.3: Monitor Execution

In the runner window, you should see:

```
2026-08-03 04:43:36Z: Running job: deploy
2026-08-03 04:43:58Z: Job deploy completed with result: Succeeded
```

The job completed successfully.

Step 9.4: Verify in GitHub Actions

Open GitHub Actions to see detailed logs:

```
https://github.com/YOUR_USERNAME/YOUR_REPO/actions
```

Click the latest workflow run to see step-by-step execution.

Step 9.5: Verify in SQL Server

Check that the SQL script executed by querying the database:

```
sqlcmd -S pmsqldba -d DBAScripts -U Github_CICD_User -P
Your_Password -C -Q "SELECT name FROM sys.tables WHERE name =
'Users'"
```

You should see the Users table listed.

10. Monitoring and Verification

Step 10.1: Monitor Deployments in Real-Time

The runner window shows real-time output. Errors appear immediately. Keep the runner window visible during deployments.

Step 10.2: Check GitHub Actions Logs

For detailed logs, visit your Actions tab. Each workflow run shows execution timestamp, each step and its status, duration, and error messages if any.

Step 10.3: Query SQL Server

Verify deployments by querying your database:

```
sqlcmd -S pmsqldba -d DBAScripts -U Github_CICD_User -P
Your_Password -C -Q "SELECT name FROM sys.tables"
```

This shows all tables created by deployments.

Step 10.4: Verify Workflow Triggers

The workflow automatically triggers when:

- SQL scripts pushed to Scripts folder on main branch
- Manual trigger via "Run workflow" button in GitHub Actions

The workflow does NOT trigger for:

- Commits to other folders
- Commits to branches other than main
- Changes to files outside Scripts folder

11. Troubleshooting Common Issues

Issue: Runner Shows "Offline" in GitHub

Symptom

Jobs stay queued in GitHub Actions. Runner shows as offline.

Solution

Check if runner process is running:

```
Get-Process | grep Runner
```

If not running, start it:

```
cd F:\runners
.\run.cmd
```

Keep the PowerShell window open. Closing it stops the runner.

Issue: Connection Fails - SSL Certificate Error

Symptom

```
SSL Provider: The certificate chain was issued by an authority
that is not trusted
```

Solution

The workflow already includes the -C flag which trusts self-signed certificates. Verify:

- 1. SQL Server is accessible:

```
sqlcmd -S pmsqldba -d DBAScripts -U Github_CICD_User -P
Your_Password -C -Q "SELECT @@VERSION;"
```

- 2. Credentials in GitHub secrets are correct.
- 3. SQL Server is listening on the correct port.

Issue: Job Fails in 3-5 Seconds

Symptom

Workflow completes immediately with failure.

Solution

Check GitHub Actions logs for specific error. Common causes:

- Scripts folder is empty
- SQL script has syntax errors
- Database name in secrets is incorrect
- SQL user lacks permissions

Test script locally:

```
sqlcmd -S pmsqldba -d DBAScripts -U Github_CICD_User -P
Your_Password -C -i Scripts\Users.sql
```

Issue: Git Merge Conflict

Symptom

```
You have not concluded your merge (MERGE_HEAD exists)
```

Solution

```
git merge --abort
git pull origin main
git push origin main
```

12. Maintenance and Best Practices

Adding New SQL Scripts

Create new SQL file in Scripts folder:

```
notepad Scripts\Orders.sql
```

Add SQL code with idempotent pattern:

```
IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'Orders')
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY IDENTITY(1,1),
    OrderName NVARCHAR(255) NOT NULL,
    CreatedDate DATETIME DEFAULT GETDATE()
)
```

Commit and push:

```
git add Scripts\Orders.sql
git commit -m "Add Orders table"
git push origin main
```

Script Naming Convention

Use numbered prefixes for execution order:

- 001-CreateUsersTable.sql
- 002-CreateOrdersTable.sql
- 003-CreateProductsTable.sql
- 004-AddStoredProcedures.sql

Scripts execute in alphabetical order. Numbering ensures correct sequence.

Script Best Practices

Write idempotent scripts that can run multiple times:

```
-- Good - Can run multiple times
IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'Users')
CREATE TABLE Users (UserID INT PRIMARY KEY)

IF EXISTS (SELECT * FROM sys.objects WHERE name = 'GetUsers')
DROP PROCEDURE GetUsers
CREATE PROCEDURE GetUsers AS SELECT * FROM Users
```

Restarting the Runner

If runner stops, restart it:

```
cd F:\runners
.\run.cmd
```

Rollback Procedure

If deployment causes issues:

1. Create rollback script:

```
notepad Scripts\Rollback-001.sql
```

2. Add recovery SQL:

```
DROP TABLE Orders
```

3. Push rollback:

```
git add Scripts\Rollback-001.sql
git commit -m "Rollback: Drop Orders table"
git push origin main
```

Summary

Your SQL Server CI/CD pipeline is now complete. Every SQL script pushed to the Scripts folder automatically deploys to your database through GitHub Actions and the self-hosted runner.

Key Components

- GitHub Repository: Version control for SQL scripts
- Workflow File: Defines automated deployment steps
- Self-Hosted Runner: Executes deployments on your Windows machine
- GitHub Secrets: Stores SQL credentials securely
- SQL Server: Receives and executes SQL scripts

The system is production-ready and fully automated.

Appendix: Complete Workflow Example

Scenario: Adding New Table to Production

Step 1: Create SQL Script

```
cd F:\GITHUB\SQL-CICD-DEMO
notepad Scripts\001-CreateNewTable.sql
```

Step 2: Write SQL

```
IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'NewTable')
CREATE TABLE NewTable (
    ID INT PRIMARY KEY IDENTITY(1,1),
    Name NVARCHAR(255) NOT NULL,
    CreatedDate DATETIME DEFAULT GETDATE()
)
```

Step 3: Test Locally

```
sqlcmd -S pmsqldba -d DBAScripts -U Github_CICD_User -P
Your_Password -C -i Scripts\001-CreateNewTable.sql
```

Step 4: Verify in SQL Server

```
SELECT * FROM NewTable
```

Step 5: Commit to Repository

```
git add Scripts\001-CreateNewTable.sql
git commit -m "Add NewTable to production"
git push origin main
```

Step 6: Monitor Deployment

Check runner window for success message, then verify in GitHub Actions logs.

Step 7: Verify in Production

```
SELECT * FROM sys.tables WHERE name = 'NewTable'
```

Table should appear in production database.